Day 33 : Docker Compose: Multi-container Basics

Task 1: Install & Verify Check if Docker Compose is available on your machine : docker compose version

If you see **command not found**, install it:`sudo apt update , sudo apt install docker-compose-plugin -y`

2.Verify the version: docker compose version

```
ubuntu@ip-172-31-44-85:~$ sudo apt install docker-compose -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-compose is already the newest version (1.29.2-6ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 51 not upgraded.
ubuntu@ip-172-31-44-85:~$ docker-compose --version
docker-compose version 1.29.2, build unknown
ubuntu@ip-172-31-44-85:~$
```

Task 2: Your First Compose File:

1.Create a folder compose-basics : mkdir compose-basics

cd compose-basics.

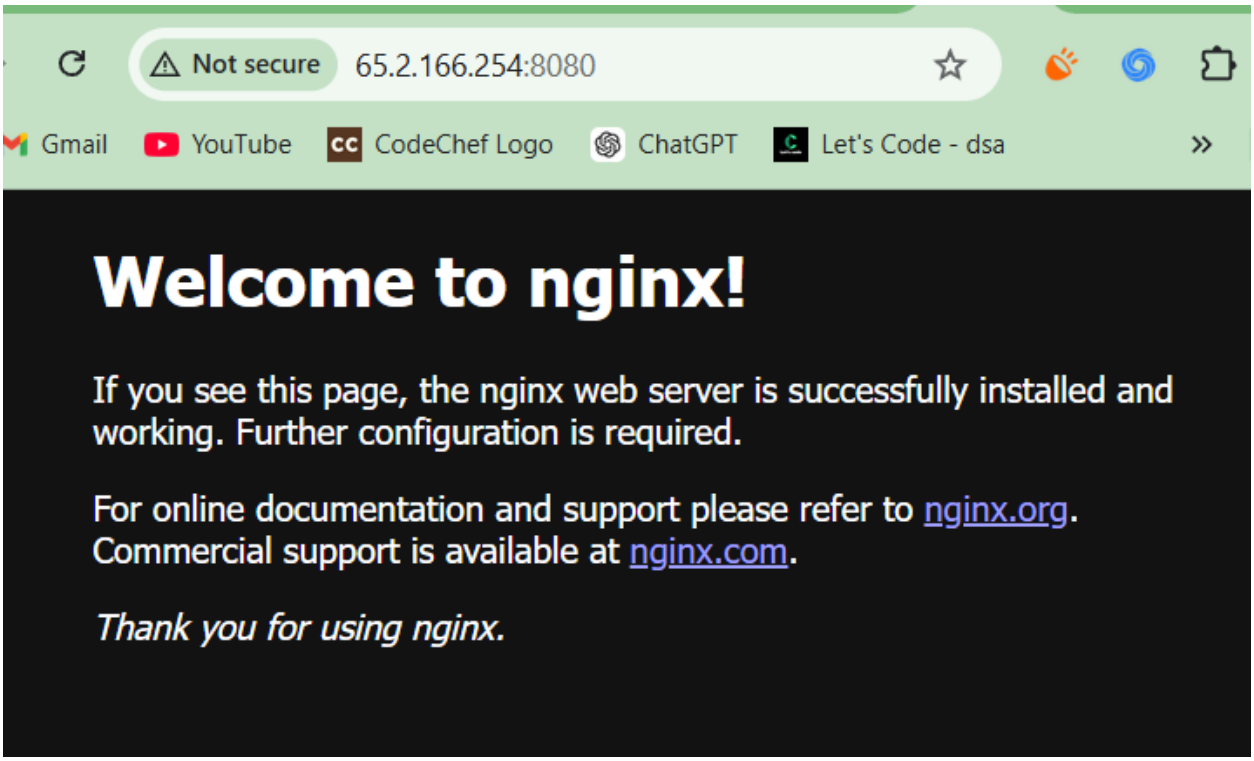
2.Write a docker-compose.yml that runs a single Nginx container with port mapping

```
version: "3"

services:

  nginx:

    image: nginx

    ports:

      - "8080:80"
```

3.Start it with docker compose up : you will see logs like - nginx | start worker processes

4.Access it in your browser: http://EC2_PUBLIC_IP:8080 , You should see the **Nginx welcome page**.

5. Stop it with docker compose down: CTRL + C and then docker compose down



Task 3: Two-Container Setup Write a docker-compose.yml that runs: A WordPress container A MySQL container They should: Be on the same network (Compose does this automatically) MySQL should have a named volume for data persistence WordPress should connect to MySQL using the service name Start it, access WordPress in your browser, and set it up. Verify: Stop and restart with docker compose down and docker compose up — is your WordPress data still there?

1. Modify compose file: vim docker-compose.yml

```
version: "3"

services:

  db:

    image: mysql:5.7

    restart: always

    environment:

      MYSQL_ROOT_PASSWORD: root123

      MYSQL_DATABASE: wordpress

      MYSQL_USER: wpuser
```

```
    MYSQL_PASSWORD: wppass
  volumes:
    - mysql-data:/var/lib/mysql
 wordpress:
  image: wordpress
  restart: always
  ports:
    - "8080:80"
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wppass
    WORDPRESS_DB_NAME: wordpress
  depends_on:
    - db
volumes:
 mysql-data:
```

2. Start application : docker-compose up –d

Check containers:`docker-compose ps`

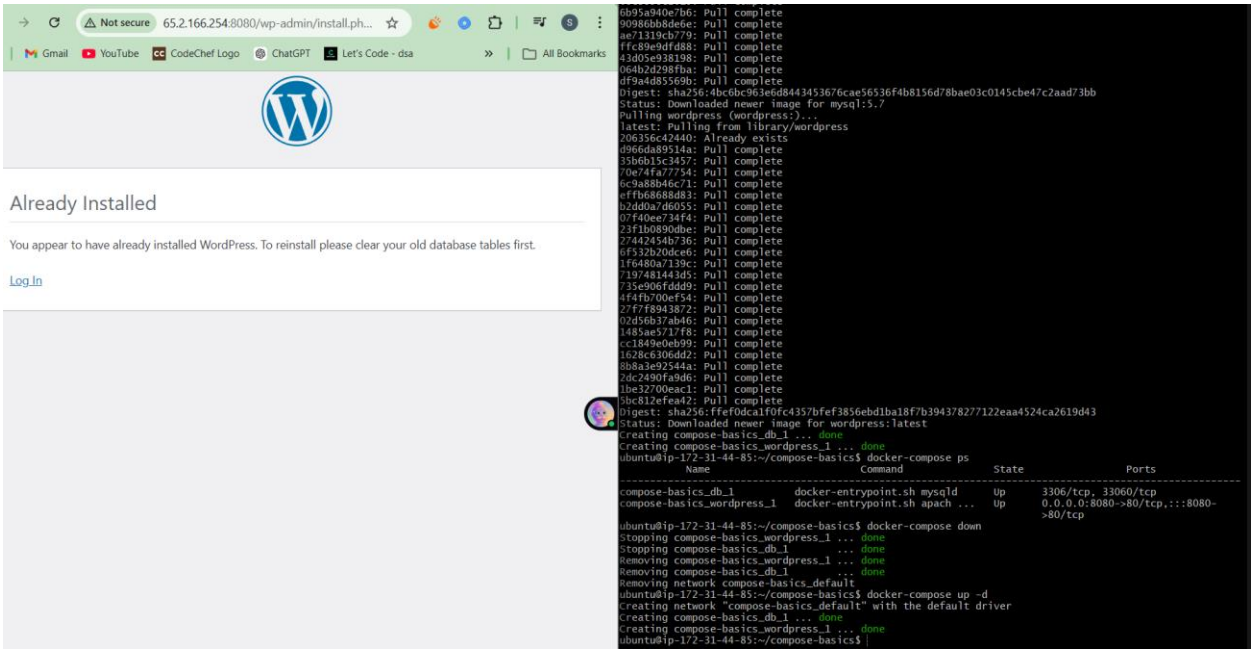
You should see:db `,wordpress`

3.Open WordPress: http://EC2_PUBLIC_IP:8080

You will see:WordPress Setup Page

4. Verify data persistence : stop: docker-compose down , Start again:docker-compose up -d

If WordPress **remembers your setup**, data persistence works.



Task 4: Compose Commands Practice and document these:

1. Start services in detached mode: docker-compose up –d

2. View running services : docker-compose ps

3.View logs of all services: docker-compose logs

4.View logs of a specific service: docker-compose logs wordpress

5. Stop services without removing : docker-compose stop

6.Remove everything (containers, networks): docker-compose down

7.Rebuild images if you make a change: docker-compose up --build

Task 5: Environment Variables

1.Add environment variables directly in your docker-compose.yml

services:

db:

```
  image: mysql:5.7
  environment:
    MYSQL_ROOT_PASSWORD: root123
    MYSQL_DATABASE: wordpress
```

Docker Compose will pass these variables to the container.

2.Create a .env file and reference variables from it in your compose file : vim .env

```
MYSQL_ROOT_PASSWORD=root123
MYSQL_DATABASE=wordpress
MYSQL_USER=wpuser
MYSQL_PASSWORD=wppass
```

3.Verify the variables are being picked up: Run:`docker-compose up -d`

docker-compose config